

Agosto 21, 2026

Patrones de Software

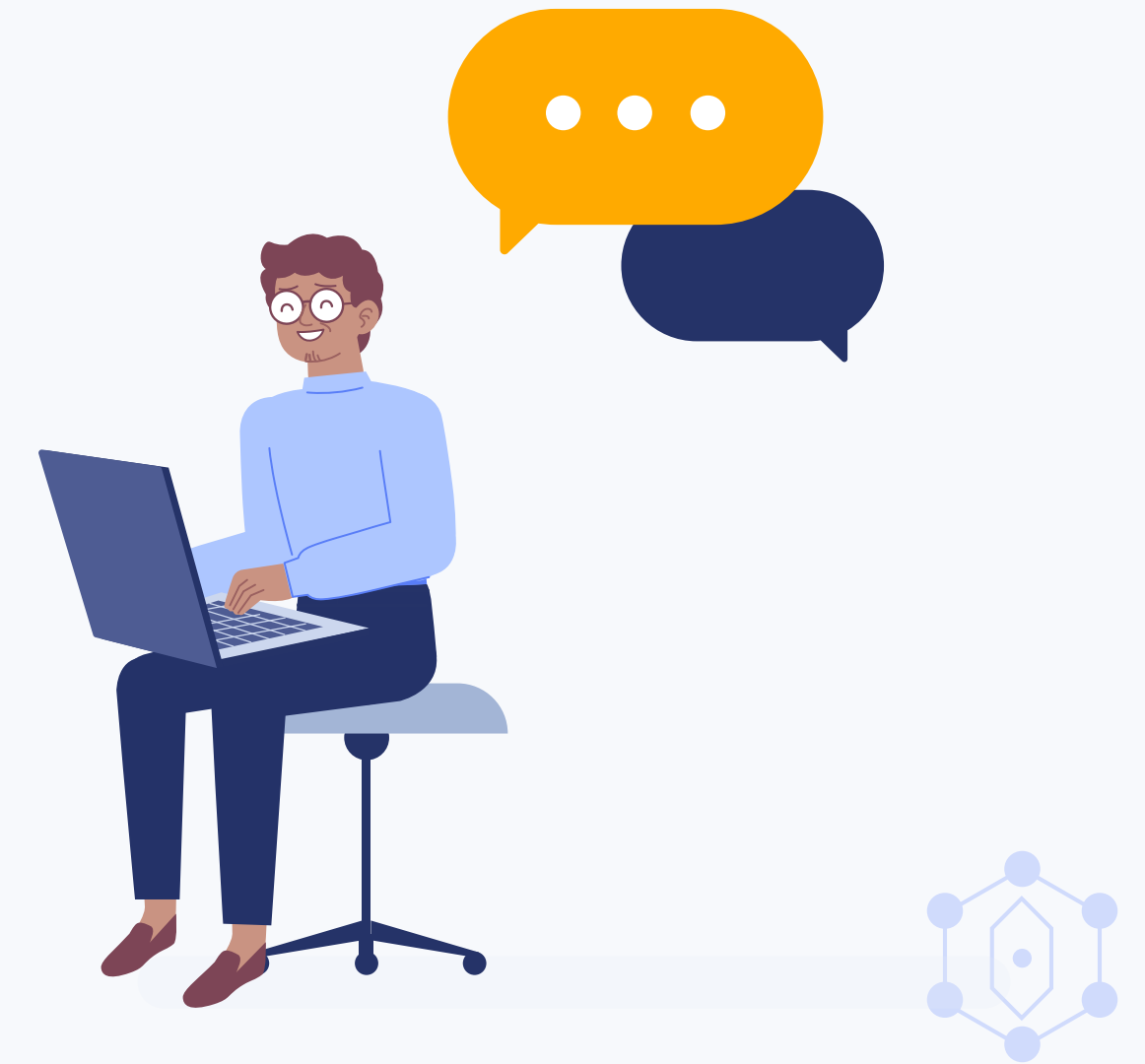
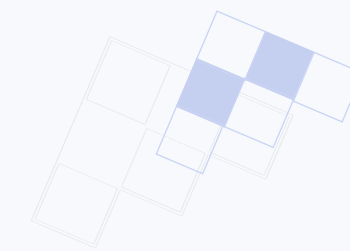
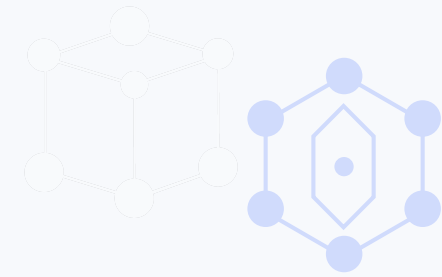
Docente: Jhonatan Andres Mideros Narvaez

Semestre: Cuarto



Recordemos

Conceptos vistos en la anterior clase



**Introducción a la
POO**

**Practica de
POO**

**Patrón
Singleton**

Patrón Prototype





Definiciones



Es un tipo de patrón **creacional** que se utiliza para crear **objetos duplicados** o **prototipos de objetos** existentes, se basa en la idea de **clonar objetos** en lugar de crearlos desde cero.

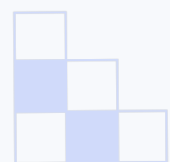


El patrón implica la **creación de un objeto prototipo inicial** que sirve como base para la creación de nuevos objetos. En lugar de crear objetos nuevos mediante la **instancia** directa de clases, se clona el **prototipo existente** y se modifica según sea necesario.



Ventajas del Patrón Prototype

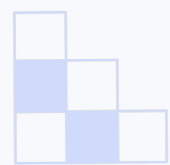
- ✓ **Reducción de la sobrecarga de creación:** En situaciones donde la creación de objetos es costosa, el patrón Prototype permite duplicar objetos ya existentes en lugar de crear nuevos desde cero, lo que puede **mejorar el rendimiento** y **reducir la carga** en el sistema.
- ✓ **Personalización eficiente:** Permite crear nuevas instancias a partir de objetos existentes y luego modificar solo las partes necesarias. Esto permite una **personalización eficiente** de objetos sin duplicar el código asociado.
- ✓ **Flexibilidad:** Los objetos prototype pueden ser **modificados en tiempo de ejecución**, lo que brinda flexibilidad en la creación y configuración de objetos.





Desventajas del Patrón Prototype

- ✓ **Complejidad de implementación:** La implementación del patrón Prototype puede ser compleja, especialmente si los **objetos contienen referencias a otros objetos**. La lógica de clonación debe ser manejada cuidadosamente para evitar problemas de referencias y estado inconsistente.
- ✓ **Costo de almacenamiento:** Si los objetos prototype son grandes y se crean muchas instancias **clonadas**, el sistema puede incurrir en un aumento en el uso de memoria debido al almacenamiento de múltiples copias del mismo prototipo.



Practica del Patrón Prototype



Online-java



<https://www.online-java.com/>

A screenshot of the Online Java IDE interface. The title bar reads "</> ONLINE JAVA BETA". Below the title bar is a toolbar with icons for file operations (folder, save, share) and settings (gear, info, settings). The main editor area shows a file named "Main.java" with a plus sign to its right. The code is as follows:

```
1
2 /*
3 Online Java - IDE, Code Editor, Compiler
4
5 Online Java is a quick and easy tool that helps you to build, compile, test your programs online.
6 */
7
8 public class Main
9 {
10     public static void main(String[] args) {
11         System.out.println("Welcome to Online IDE!! Happy Coding :)");
12     }
13 }
14
```

The status bar at the bottom indicates "Ln: 14, Col: 1".

Código de la clase

(Figuras 1)

```
1  import java.util.*;
2
3  // Interface Prototype
4  interface Shape {
5      Shape clone();
6      void draw();
7  }
```

Código de la clase (Figuras 2)

```
9  // Class implementing prototype shape
10 class Circle implements Shape {
11     private int radius;
12
13     public Circle(int radius) {
14         this.radius = radius;
15     }
16
17     public void setRadius(int radius) {
18         this.radius = radius;
19     }
20
21     @Override
22     public Shape clone() {
23         return new Circle(this.radius);
24     }
25
26     @Override
27     public void draw() {
28         System.out.println("Drawing a circle with radius " + radius);
29     }
30 }
```

Código de la clase (Figuras 3)

```
32 public class Main {  
33     public static void main(String[] args) {  
34         // create object of class circle  
35         Shape originalCircle = new Circle(5);  
36  
37         // clone object from "originalCircle"  
38         Shape clonedCircle = originalCircle.clone();  
39  
40         // draw from object cloned  
41         clonedCircle.draw();  
42  
43         // Modify radius from object cloned  
44         ((Circle) clonedCircle).setRadius(8);  
45         clonedCircle.draw();  
46     }  
47 }
```

Patrón Builder





Definiciones



Es un patrón **creacional** que se utiliza para **construir objetos complejos** con múltiples partes y configuraciones de manera flexible y clara. Este patrón se usa especialmente cuando se requiere construir objetos con **muchas propiedades opcionales** o **muchas configuraciones**.

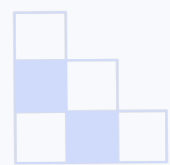


Este patrón **separa** la creación de un objeto complejo de su representación, permitiendo que el mismo proceso de construcción **pueda crear diferentes representaciones**.



Ventajas del Patrón Builder

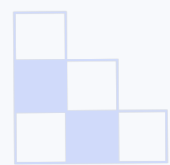
- ✓..... **Flexibilidad en la construcción:** permite construir objetos complejos, ajustando configuraciones y propiedades en el camino.
- ✓..... **Separación de responsabilidades:** El código de construcción se separa del código de la representación del objeto, lo que mejora la claridad y mantenibilidad del código.
- ✓..... **Reusabilidad:** Los mismos componentes de construcción pueden utilizarse para crear diferentes representaciones de objetos. Esto facilita la reutilización de la lógica de construcción.





Desventajas del Patrón Builder

- ✓ **Complejidad adicional:** La implementación del patrón Builder puede resultar en la creación de varias clases adicionales, lo que puede aumentar la complejidad del código en comparación con la creación directa de objetos.
- ✓ **Necesidad de diseño:** Se debe diseñar cuidadosamente la estructura de clases y las interfaces para garantizar que el patrón Builder se implemente correctamente.



Practica del Patrón Builder



Online-java



<https://www.online-java.com/>

A screenshot of the Online Java IDE interface. The title bar at the top reads "</> ONLINE JAVA BETA". Below the title bar is a toolbar with icons for file operations (folder, save, share) and settings (gear, info, settings). The main editor area shows a file named "Main.java" with a plus sign to its right. The code in the editor is as follows:

```
1
2 /*
3 Online Java - IDE, Code Editor, Compiler
4
5 Online Java is a quick and easy tool that helps you to build, compile, test your programs online.
6 */
7
8 public class Main
9 {
10     public static void main(String[] args) {
11         System.out.println("Welcome to Online IDE!! Happy Coding :)");
12     }
13 }
14
```

The status bar at the bottom left indicates "Ln: 14, Col: 1".

Código de la clase (Computer - 1)

```
1  class Computer {
2      private String cpu;
3      private String memory;
4      private String storage;
5      private String gpu;
6
7      public void setCpu(String cpu) {
8          this.cpu = cpu;
9      }
10
11     public void setMemory(String memory) {
12         this.memory = memory;
13     }
14
15     public void setStorage(String storage) {
16         this.storage = storage;
17     }
18
19     public void setGpu(String gpu) {
20         this.gpu = gpu;
21     }
22
23     public void showComputer() {
24         System.out.println("Computer with CPU: "
25             + cpu + ", Memory: "
26             + memory + ", Storage: "
27             + storage + ", GPU: " + gpu);
28     }
29 }
```

Código de la clase (Computer - 2)

```
31 interface ComputerBuilder {  
32     void buildCpu();  
33     void buildMemory();  
34     void buildStorage();  
35     void buildGpu();  
36     Computer getComputer();  
37 }
```

Código de la clase (Computer - 3)

```
39 class GamingComputerBuilder implements ComputerBuilder {
40     private Computer computer = new Computer();
41
42     @Override
43     public void buildCpu() {
44         computer.setCpu("Intel Core i7");
45     }
46
47     @Override
48     public void buildMemory() {
49         computer.setMemory("16GB DDR4 RAM");
50     }
51
52     @Override
53     public void buildStorage() {
54         computer.setStorage("512GB SSD + 1TB HDD");
55     }
56
57     @Override
58     public void buildGpu() {
59         computer.setGpu("NVIDIA RTX 3080");
60     }
61
62     @Override
63     public Computer getComputer() {
64         return computer;
65     }
66 }
```

Código de la clase (Computer - 4)

```
68  class ComputerDirector {
69      private ComputerBuilder computerBuilder;
70
71      public ComputerDirector(ComputerBuilder computerBuilder) {
72          this.computerBuilder = computerBuilder;
73      }
74
75      public void buildComputer() {
76          computerBuilder.buildCpu();
77          computerBuilder.buildMemory();
78          computerBuilder.buildStorage();
79          computerBuilder.buildGpu();
80      }
81  }
```

Código de la clase (Computer - 5)

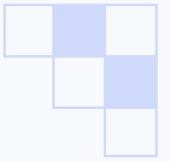
```
68  class ComputerDirector {
69      private ComputerBuilder computerBuilder;
70
71      public ComputerDirector(ComputerBuilder computerBuilder) {
72          this.computerBuilder = computerBuilder;
73      }
74
75      public void buildComputer() {
76          computerBuilder.buildCpu();
77          computerBuilder.buildMemory();
78          computerBuilder.buildStorage();
79          computerBuilder.buildGpu();
80      }
81  }
```

Código de la clase (Computer - 6)

```
83 public class Main {  
84     public static void main(String[] args) {  
85         ComputerBuilder gamingComputerBuilder = new GamingComputerBuilder();  
86         ComputerDirector director = new ComputerDirector(gamingComputerBuilder);  
87  
88         director.buildComputer();  
89  
90         Computer computer = gamingComputerBuilder.getComputer();  
91         computer.showComputer();  
92     }  
93 }
```

Taller sobre Patrón Prototype y Builder





Taller Patrón Prototype y Builder aplicado en un caso de estudio real

- ✓..... Seleccione un caso de estudio de la **vida real** donde aplique los patrones **Prototype** y **Builder**. El script debe ser realizado en **java**.

